

ASSIGNMENT 3: DYNAMIC PROGRAMMING

Instructor: Orhan Özgüner

Due: Wednesday, November 5 before 11:59 PM

Provide a dynamic programming solution to the following questions. You do not have to follow these steps, but they may be helpful in thinking of a solution:

Here are guided steps to help you while thinking about a solution:

- (i) Identify the “last” question you need to answer in developing a solution.
- (ii) Define and prove optimal substructure.
- (iii) Define subproblems, express the solution to the overall problem in terms of the subproblems.
- (iv) Formulate a recursive solution to the subproblems. Do not forget to specify the base case(s).
- (v) Characterize the runtime of the resulting procedure assuming that you would implement your solution using a bottom-up procedure.
- (vi) Provide the pseudo code of the bottom-up procedure you use to compute the value of the optimal solution, as well as the procedure for reconstructing the optimal solution.

Problem 1

We are given n types of coin denominations with integer values $v_1, v_2, \dots, v_n$. Given an integer t , we would like to compute the minimum number of coins to make change for t (i.e., we would like to compute the minimum number of coins that add up to t , where repetitions are allowed). We know that one of the coins has value 1, so we can always make change for any amount of money t . For example, if we have coin denominations of 1, 2, and 5, then the optimal solution for $t = 9$ is 5, 2, 2.

Problem 1 Solution

1: The Last Decision

The last decision to be made is the final denomination to be added so that all previously added denominations sum to a value of t .

2: Subproblem Definition

The subproblems are defined as the minimum number of coins of the considered denominations to make change for a value k where $k \leq t$, denoted as $u[k]$. The optimal solution to the entire problem is given by $u[t] = \min_{1 \leq \ell \leq n} (u[t - v_\ell] + 1)$, where v_ℓ in $\min_{1 \leq \ell \leq n} (u[t - v_\ell])$ is the last required denomination to make change for t with the minimum number of coins.

3: Recurrence Relation

$$u[k] = \begin{cases} 1 & \text{if } \exists j \text{ such that } v_j = k \\ \min_{1 \leq \ell \leq n} (u[k - v_\ell]) + 1 & \text{otherwise} \end{cases} \quad (1)$$

4: Pseudocode

Algorithm 1 Bottom-Up Implementation to Compute Value of Optimal Solution

```

1: function COMPUTEVALUEOFOPTIMALSOLUTION( $V, t$ )
2:    $u, C \leftarrow$  arrays over  $[0..t]$  filled with  $\infty$ 
3:    $\triangleright u$  is an array where  $u[i]$  is the minimum number of coins needed to create  $i$ 
4:    $\triangleright C$  is an array where  $C[i]$  is the last coin added to make a value of  $i$ .
5:    $u[0] \leftarrow 0$ 
6:   for  $i \leftarrow 1$  to  $t$  do
7:     for  $\ell \leftarrow 1$  to  $\text{length}(V)$  do
8:       if  $v[\ell] \leq i$  and  $u[i - v[\ell]] + 1 < u[i]$  then
9:          $u[i] \leftarrow u[i - v[\ell]] + 1$ 
10:         $C[i] \leftarrow v[\ell]$ 
11:       end if
12:     end for
13:   end for
14:   return  $u[t], u, C$ 
15: end function

```

Algorithm 2 Procedure To Reconstruct Optimal Solution

```

1: function RECONSTRUCTOPTIMALCHANGE( $u, C, t$ )
2:   Let  $D$  be an array over  $[1..u[t]]$ 
3:    $ptr \leftarrow t$ 
4:   for  $j \leftarrow u[t]$  down to 1 do
5:      $D[j] \leftarrow C[ptr]$ 
6:      $ptr \leftarrow ptr - C[ptr]$ 
7:   end for
8:   return  $D$ 
9: end function

```

5: Proof

We will prove by induction that the dynamic programming algorithm correctly computes the minimum number of coins $u[t]$ needed to form any non-negative integer t .

Base Case:

The base case is $u[0] = 0$, as no coins are needed to make a sum of 0.

Inductive Step:

Assume that $u[k]$ is correct for all $k < t$. We need to show that $u[t]$ is also correct.

The recurrence relation used by the algorithm is

$$u[t] = \min_{v_\ell \leq t} (u[t - v_\ell] + 1).$$

Given our inductive hypothesis, $u[k]$ is the minimum number of coins needed to form k for all $k < t$. If we add one more coin v_ℓ to $u[t - v_\ell]$, we form t , and $u[t - v_\ell] + 1$ is the minimum number of coins needed to form t if v_ℓ is included in the optimal solution for t .

By taking the minimum over all $v_\ell \leq t$, we ensure that $u[t]$ is the minimum number of coins needed to form t .

Thus, by induction, the algorithm correctly computes $u[t]$ for all t .

Proof of Optimal Substructure

We next prove that the optimal solution to our problem contains within it optimal solutions to subproblems, which is the principle of optimality for dynamic programming.

Lemma: If $u[t]$ gives the minimum number of coins needed for any total t , then for any t and v_ℓ , $u[t] = u[t - v_\ell] + 1$ implies that $u[t - v_\ell]$ is also optimal.

Proof: Assume for contradiction's sake, that $u[t]$ is optimal but $u[t - v_\ell]$ is not. This would mean there exists a better solution to $t - v_\ell$, which we will call $u'[t - v_\ell]$, which uses fewer coins than $u[t - v_\ell]$.

If $u'[t - v_\ell]$ is indeed better, then we would have $u'[t - v_\ell] + 1 < u[t - v_\ell] + 1$, which contradicts the optimality of $u[t]$ because $u[t]$ could be improved by using $u'[t - v_\ell] + 1$.

This contradiction implies that $u[t - v_\ell]$ must be optimal. Therefore, the solution to any subproblem used in constructing the optimal solution to the problem is itself optimal, satisfying the principle of optimal substructure.

6: Runtime Analysis

There are $\Theta(t)$ possible subproblems in the form of possible lesser values of t , to build our final solution from the bottom up. We must calculate all possible minimal values up to t (which will be possible for every positive integer as 1 is always a denomination) to ensure that $k - v_\ell$ will be available for every k and every v_ℓ .

There are $\Theta(n)$ denominations to consider per subproblem, so the overall runtime is $\Theta(nt)$.

Problem 2

Given two strings $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, the edit distance between X and Y is defined as the minimum number of edit operations (*replacement*, *insertion*, or *deletion* of a character) required to convert X to Y . For example, the edit distance between $X = \text{esteban}$ and $Y = \text{stephen}$ is 4, comprising of 1 deletion (e), 1 insertion (h), and 2 replacements ($b \rightarrow p$ and $a \rightarrow e$). We would like to compute the edit distance between two given strings.

Problem 2 Solution

1: The Last Decision

We will solve this problem from the beginning of the strings X and Y , to the end. The last decision we make is, then, the last edit (in the sense of proximity to the end of the string) needed to turn X into Y , in the optimal sequence of edits.

2: Subproblems

We will define a two-dimensional array D , representing all possible sequences of edits that could turn X into Y . Specifically, $D[i, j]$ is the minimum cost to transform the i th character of X , into the j th character of Y . This can be done by inserting or deleting characters in front of the i th character of X to “move it forwards” or “backwards” and match a character in Y , or by substitution.

3: Recurrence Relation

$$D[i, j] = \begin{cases} i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ \min \begin{pmatrix} D[i-1, j] + 1, \\ D[i, j-1] + 1, \\ D[i-1, j-1] + \delta(i, j) \end{pmatrix} & \text{otherwise} \end{cases} \quad (2)$$

where $\delta(i, j) = 0$ if $x_i = y_j$, and 1 otherwise.

4: Pseudocode

Algorithm 3 Compute Edit Distance

```
1: function EDITDISTANCE( $X, Y$ )
2:    $m \leftarrow \text{length}(X)$ 
3:    $n \leftarrow \text{length}(Y)$ 
4:    $D \leftarrow \text{new } m \times n \text{ array}$ 
5:   for  $i \leftarrow 0$  to  $m$  do
6:      $D[i, 0] \leftarrow i$ 
7:   end for
8:   for  $j \leftarrow 0$  to  $n$  do
9:      $D[0, j] \leftarrow j$ 
10:  end for
11:  for  $i \leftarrow 1$  to  $m$  do
12:    for  $j \leftarrow 1$  to  $n$  do
13:      if  $X[i] = Y[j]$  then
14:         $cost \leftarrow 1$ 
15:      else
16:         $cost \leftarrow 0$ 
17:      end if
18:       $D[i, j] \leftarrow \min(D[i-1, j] + 1, D[i, j-1] + 1, D[i-1, j-1] + cost)$ 
19:    end for
20:  end for
21:  return  $D[m, n]$ 
22: end function
```

5: Proof

First, we will prove the correctness of the algorithm inductively:

Base Cases:

- When $j = 0$, $D(X, \epsilon) = i$. This is because we must perform i deletions to transform X into an empty string.
- When $i = 0$, $D(\epsilon, Y) = j$. This is because we must perform j insertions to transform an empty string into Y .

Inductive Step: Assume for strings shorter than m and n , the edit distances are optimally computed. For the final character, we then consider:

1. Deletion from X , at cost 1: $D[x_{m-1}, y_{n-1}] + 1$.
2. Insertion to X from Y , at cost 1: $D[x_m, y_{n-1}] + 1$.
3. Either substitution, or nothing at all:
 - If $x_m = y_n$, then $D[x_{m-1}, y_{n-1}]$ (no additional cost).
 - If $x_m \neq y_n$, then $D[x_{m-1}, y_{n-1}] + 1$ (substitution cost)

6: Runtime Analysis

Each subproblem $D[i, j]$ is evaluated in constant time. With $m \times n$ of these subproblems to solve, the overall time complexity is $\Theta(mn)$.

Problem 3

We are given n currencies and an exchange rate r_{ij} for any pair of currencies i and j . Namely, if we exchange 1 unit of currency i with currency j , we receive r_{ij} units of currency j . If we are given a source currency s and a target currency t , then we can go through a path of different currencies to reach t from s so as to maximize our profit. The markets can also charge an exchange fee depending on the number of exchanges we make. For example if the exchange fee is $f(k)$ for making k exchanges and we start with 1 unit of currency s , then the path of exchanges $s \rightarrow t$ will yield $r_{st} - f(1)$ units of currency t , whereas the path of exchanges $s \rightarrow i \rightarrow j \rightarrow t$ will yield $r_{si} \times r_{ij} \times r_{jt} - f(3)$ units of currency t . The problem is to find the sequence of exchanges that will maximize the amount of target currency t we can obtain for a given source currency s .

Problem 3 Solution

1: The Last Decision

We will work through this problem by gradually expanding the pool of vertices we consider, in the order in which the vertices are set out in v , and determine whether the most profitable path includes each. Thus, the very last decision we will make, will be whether the path includes vertex v_n or not.

2: Subproblems

We will define the three-dimensional array M as the most profitable path from vertex $v[i]$ to vertex $v[j]$, using only the set of vertices v_1 through v_k .

3: Recurrence Relation

$$M[i, j, k] = \begin{cases} 1 & \text{if } k = 0 \text{ and } i = j \\ j & \text{if } k = 0 \text{ but } i \neq j \\ \max \left(\begin{array}{l} M[i, j, k-1] + 1, \\ M[i, k, k-1] \times M[k, j, k-1], \end{array} \right) & \text{otherwise} \end{cases} \quad (3)$$

4: Pseudocode

Algorithm 4 Currency Arbitrage Algorithm

```
1: function ARBITRAGE( $v, r(v_1, v_2), n$ )
2:    $M, P \leftarrow n \times n \times n$  arrays
3:   for  $i \leftarrow 1$  to  $n$  do
4:     for  $j \leftarrow 1$  to  $n$  do
5:       if  $i = j$  then
6:          $M[i, j, 0] \leftarrow 1$ 
7:       else
8:          $M[i, j, 0] \leftarrow r(i, j)$ 
9:       end if
10:    end for
11:  end for
12:  for  $k \leftarrow 1$  to  $n$  do
13:    for  $i \leftarrow 1$  to  $n$  do
14:      for  $j \leftarrow 1$  to  $n$  do
15:        if  $M[i, k, k-1] \times M[k, j, k-1] > M[i, j, k-1]$  then
16:           $M[i, j, k] \leftarrow M[i, k, k-1] \times M[k, j, k-1]$ 
17:           $P[i, j, k] \leftarrow \mathbf{TRUE}$ 
18:        else
19:           $M[i, j, k] \leftarrow M[i, j, k-1]$ 
20:           $P[i, j, k] \leftarrow \mathbf{FALSE}$ 
21:        end if
22:      end for
23:    end for
24:  end for
25:  for  $i \leftarrow 1$  to  $n$  do
26:    if  $M[i, i, n] > 1$  then
27:       $s \leftarrow \text{FIND\_PATH}(P, v, i, i, n)$ 
28:       $s \leftarrow v[i] \oplus s \oplus v[i]$   $\triangleright$  Concatenate  $v[i]$  to both ends of  $s$ 
29:      return  $(M(i, i, 0), s)$ 
30:    end if
31:  end for
32:  return "No Solution"
33: end function
```

Algorithm 5 Optimal Path Determination Algorithm

```
1: function FIND_PATH( $P, v, i, j, k$ )
2:   if  $k = 0$  then
3:     return Empty Path
4:   end if
5:   if  $P(i, j, k)$  is TRUE then
6:     return FIND_PATH( $P, v, i, k, k - 1$ )  $\oplus v[k]$   $\oplus$  FIND_PATH( $P, v, k, j, k - 1$ )  $\triangleright$  Concatenate
      paths to and from  $v[k]$ , with  $v[k]$ .
7:   else
8:     return FIND_PATH( $P, v, i, j, k - 1$ )
9:   end if
10: end function
```

5: Proof

We will prove the correctness of the algorithm using an inductive proof.

Base Case: There are two base cases to consider for paths of length $k = 0$ (discounting the starting and ending node):

If $i \neq j$, then the starting and ending nodes are different, and the maximal cost can only be the cost of traveling between them: $r(i, j)$.

If $i = j$, then the starting and ending nodes are the same, and no travel is possible. So, the maximum possible cost is always 1.

In either case, $M[i, j, 0]$ is assigned the correct value for all i and j .

Inductive Step: Assume that $M(i, j, k - 1)$ stores the maximum profit attainable by traveling from i to j , for all currencies $i, j < n$, using only the currencies v_1 through v_{k-1} . Then, consider $M(i, j, k)$.

It is now possible for the optimal solution to traverse node v_k . This can result in one of two outcomes:

1. The most profitable path from i to j includes v_k . That means the maximum profit is the product of the most profitable path between i and k , and the path between k and j : $M[i, j, k]M[i, k, k - 1] \times M[k, j, k - 1]$.
2. The most profitable path from i to j does not include v_k . That means the maximum profit is unchanged, $M[i, j, k] = M[i, j, k - 1]$.

Since there are only two possible outcomes for the addition of node v_k , and the program assigns the maximum of the two options to $M[i, j, k]$, this shows that $M[i, j, k]$ stores the maximum possible profit for a path between i and j containing only nodes v_1 through v_k .

6: Runtime Analysis

The runtime of this algorithm is $\Theta(n^3)$, where n is the number of nodes in the graph. The first loop, covering the base cases, runs for n^2 constant-time iterations. Then, the second loop, filling in the rest of the table, runs for n^3 constant-time iterations. Then, we loop and check each node of the table to see if it contains a cycle. This takes n iterations at most. Then, we reconstruct the path, which contains at most n constant-time recursive calls (as each vertex is checked in the

worst case to see if it belongs to the path). If the path contains every node, the for-loop used to concatenate the nodes together will also run in $\Theta(n)$ time.

Therefore, the overall runtime is dominated by the assembly of M , taking $\Theta(n^3)$ time.